DISEÑO DE INTERFACES DE USUARIO

Introducción a React para proyectos Frontend

Profesor Alex Arenas
Frontend Developer



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

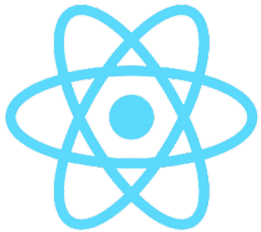
DEPARTAMENTO
DE INFORMÁTICA

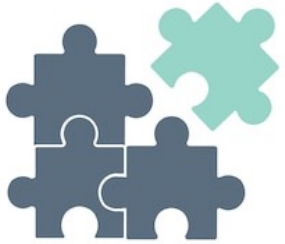
Agenda

1. React
2. BEM
3. Webpack
4. Proyecto de ejemplo
   - Prototipo de baja fidelidad
   - Estructura de componentes
   - Estructura de archivos del proyecto
   - Componentes y ruteador
   - Componentes con estado

React

- Es una de las Librerías más populares para desarrollar interfaces, no es un Framework
- Desarrollado por Facebook
- Su paradigma es Programación orientada a componentes
  - Cada fragmento de la interfaz es un Componente
  - Un componente contiene código Javascript, HTML y CSS





https://react.dev/learn

BEM

- Significa **B**lock **E**lement **M**odifier

- Es una metodología para crear componentes reutilizables y que puedan ser fácilmente entendibles a nivel de CSS
- Define reglas para nombrar a los componentes y sus variaciones

Classic CSS

```css
#opinions_box h1 {
    margin: 0 0 8px 0;
    text-align: center;
}

#opinions_box {
    p.more_pp {
        a {
            text-decoration: underline;
        }
    }

    input[type="text"] {
        border: 1px solid #ccc!important;
    }
}
```

BEM

```css
.opinions-box {
    margin: 0 0 8px 0;
    text-align: center;

    &__view-more {
        text-decoration: underline;
    }

    &__text-input {
        border: 1px solid #ccc;
    }

    &--is-inactive {
        color: gray;
    }
}
```

https://getbem.com/introduction/

Webpack

- Es un empaquetador de código para generar todos los archivos necesarios para que un proyecto funcione
  - Permite unificar todas las dependencias de un proyecto (librerías y assets principalmente) y transformarlas a un único archivo que ejecutará el proyecto completo
  - Su modo desarrollo permite agilizar el desarrollo porque soluciona el problema de servir archivos estáticos como imágenes



https://webpack.js.org/concepts/

Proyecto de ejemplo

- Vamos a realizar el siguiente proyecto

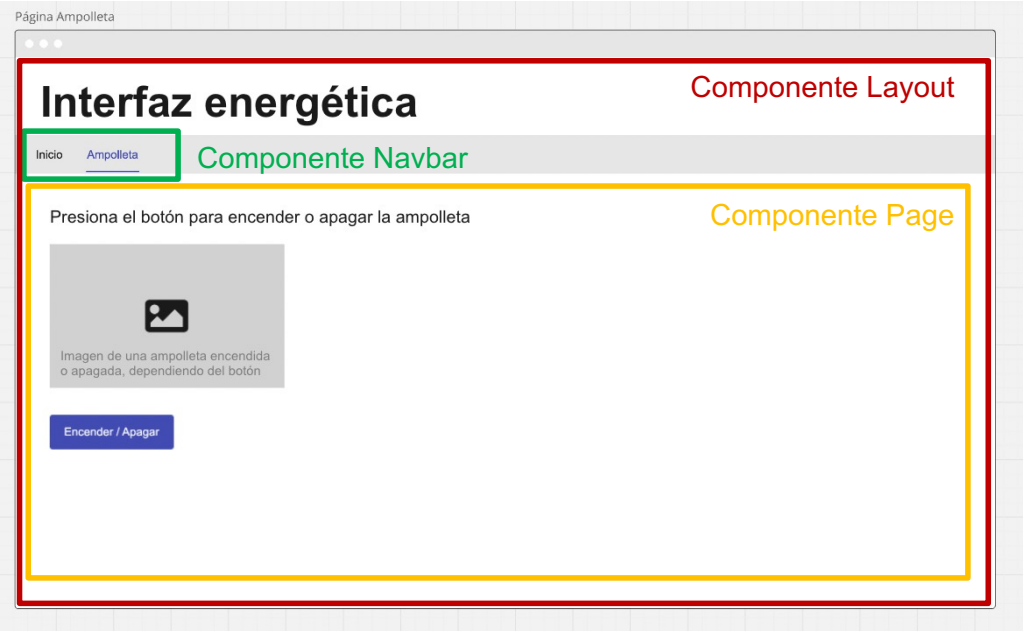




Prototipo de baja fidelidad realizado en Miro

Proyecto de ejemplo

- Definición de la estructura de componentes




```
<Layout>
    <h1>Interfaz energética</h1>
    <Navbar />
    <Page />
</Layout>
```

Proyecto de ejemplo

- El código fuente del proyecto completo se encuentra en el repositorio

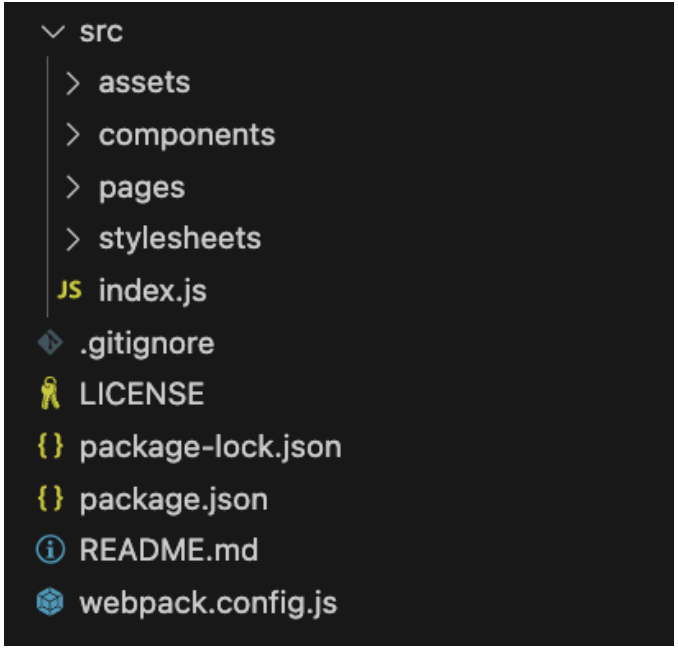
  https://github.com/alexarenasf/frontend-sample-app/

- A continuación, veremos cómo fue desarrollado paso a paso
- Una vez clonado el repositorio, lo primero es ejecutar el siguiente comando para instalar las dependencias

```
> npm install
```

Proyecto de ejemplo

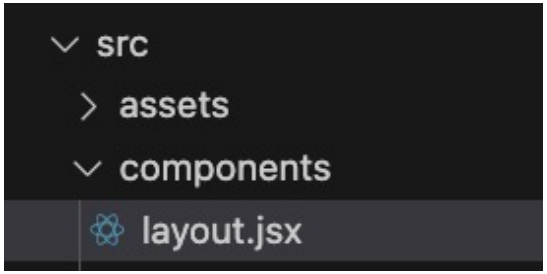
- Estructura de los archivos del proyecto



Directorio / Archivo	Para que será usado
src/assets	Archivos estáticos, imágenes
src/components	Componentes react que son un fragmento de una página
src/pages	Componentes react que son página
src/stylesheets	Estilos css y scss
src/index.js	Archivo principal del proyecto
package.json	Archivo que contiene información del proyecto y dependencias instaladas
webpack.config.js	Configuración de webpack

Proyecto de ejemplo

- Componente Layout



React utiliza la extensión jsx que consiste en un archivo que contiene código **javascript** y **html** en el mismo archivo

```
import React from 'react'

const Layout = () => {
      return (
      <div className='layout'>
            <h1 className='layout__title'>
            Interfaz energética
            </h1>
            {/* Add NavBar here */}
            <div className='layout__page'>
                  {/* Add Page here */}
                  El contenido de la página va aquí
            </div>
      </div>
      )
}

export default Layout
```

Proyecto de ejemplo

- Agregando el componente al archivo principal del proyecto

src/index.js

```
import React from 'react'
import { createRoot } from 'react-dom/client';

import Layout from './components/layout'

document.body.innerHTML = '<div id="root"></div>';
const root = createRoot(document.getElementById('root'));
root.render(<Layout />);
```

Proyecto de ejemplo

- Para ejecutar el proyecto en el navegador, escribimos en la terminal el siguiente comando que iniciará Webpack en modo desarrollo

```
> npm run dev
```

- Se desplegará el proyecto en http://localhost:5050/

- El modo desarrollo permite que el navegador se actualice automáticamente cada vez que realicemos cambios en el código

Proyecto de ejemplo

- Si ejecutamos el proyecto ahora esto es lo que veremos



- Así que vamos a agregar algunos estilos css

Proyecto de ejemplo

- Componente Layout (estilos)

```
import React from 'react'

const Layout = () => {
        return (
        <div className='layout'>
                <h1 className='layout__title'>
                Interfaz energética
                </h1>
                {/* Add NavBar here */}
                <div className='layout__page'>
                        {/* Add Page here */}
                        El contenido de la página va aquí
                </div>
        </div>
        )
}

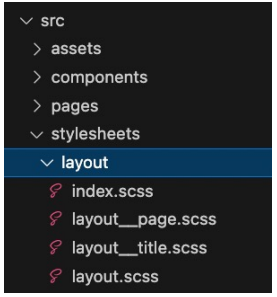
export default Layout
```

Usando **BEM**, hemos definido nombres para los elementos HTML del componente en los atributos className

El Bloque principal se llama **layout**
Dentro hay dos elementos:

- El elemento **layout__title**
- El elemento **layout__page**

Así que vamos a crear los siguientes archivos para darle estilo al bloque principal y a sus elementos



Proyecto de ejemplo

- Componente Layout (estilos)

Corregiremos algunos márgenes para que el texto no se vea tan pegado a los bordes con los siguientes estilos

src > stylesheets > layout > layout__title.scss >

```scss
.layout__title {
  font-size: 24pt;
  margin: 0;
  padding: 20px;
}
```

src > stylesheets > layout > layout__page.scss >

```scss
.layout__page {
  padding: 20px;
}
```

src > stylesheets > layout > layout.scss

```scss

```

src > stylesheets > layout > index.scss

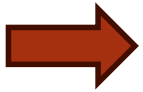
```scss
@import './layout.scss';
@import './layout__page.scss';
@import './layout__title.scss';
```

Proyecto de ejemplo

- Cargando los estilos en el archivo principal del proyecto

  Crearemos un archivo principal de estilos para ir cargando los estilos de los componentes y hacer otros cambios globales

src/stylesheets/index.scss



```
@import
url('https://fonts.googleapis.com/css2?family=Open+Sans&display=swap');

@import './layout/index.scss';

html, body {
     padding: 0;
     margin: 0;
}

* {
     font-family: 'Open Sans', sans-serif;
}
```

Proyecto de ejemplo

- Cargando los estilos en el archivo principal del proyecto

  Hay que agregar el archivo principal de estilos en el archivo principal del proyecto

src/index.js



```
import React from 'react'
import { createRoot } from 'react-dom/client';

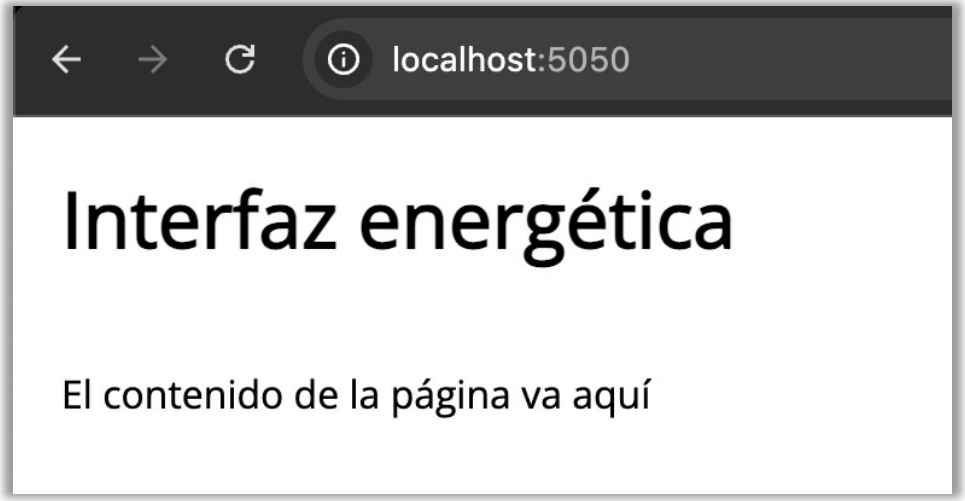
import Layout from './components/layout'

import './stylesheets/index.scss'

document.body.innerHTML = '<div id="root"></div>';
const root = createRoot(document.getElementById('root'));
root.render(<Layout />);
```

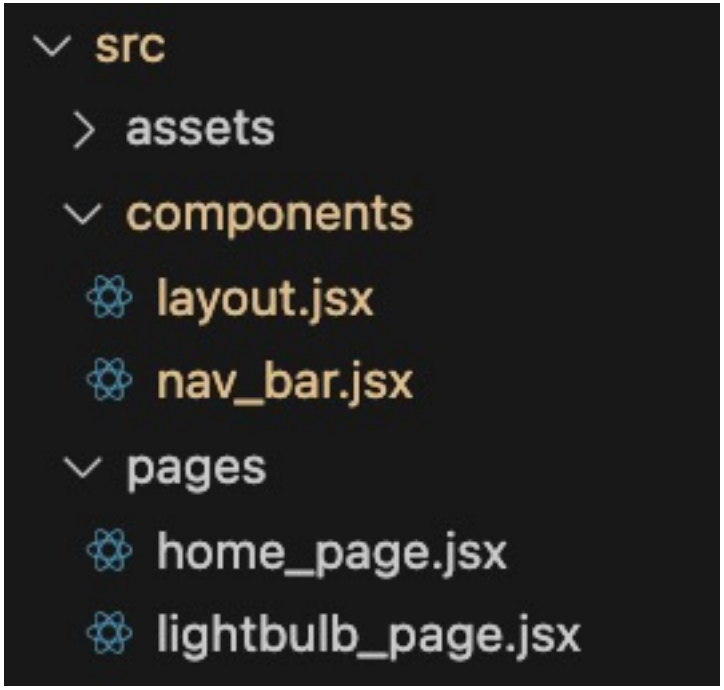
Proyecto de ejemplo

- Ahora el proyecto se ve así



Proyecto de ejemplo

- Vamos a hacer 3 componentes más
  - El componente NavBar con la barra de navegación
  - Un componente para la página Inicio
  - Un componente para la página Ampolleta
- Vamos a agregar un ruteador para mostrar una de las dos páginas



Proyecto de ejemplo

- Componente Página Inicio

src/pages/home_page.jsx

```
import React from 'react'

export const HomePage = () => {
     return (
     <>
          <p>¡Te damos la bienvenida a la Interfaz
          energética!</p>
          <p>En esta página encontraras una 💡 ampolleta
          que podrás encender y apagar con un botón</p>
     </>
     )
}

export default HomePage
```

Proyecto de ejemplo

- Componente Página Ampolleta

src/pages/lightbulb_page.jsx

```
import React from 'react'
import LightBulbOff from '../ assets/light-off.png '

export const LightbulbPage = () => {
      return (
      <div className='lightbulb-page'>
            <p>Presiona el botón para encender o apagar la ampolleta</p>
            <img src={LightBulbOff} height={130} alt='ampolleta' />
            <br />
            <br />
            <button onClick={() => {}}>
                        Encender
            </button>
      </div>
      )
}

export default LightbulbPage
```



assets/light-off.png

Webpack se encarga de que las imágenes estén disponibles al importarlas y usarlas como variables

Proyecto de ejemplo

- Componente NavBar
  - De acuerdo a **BEM**, está formado por el bloque principal **nav-bar**
  - Tiene dos elementos de tipo **nav-bar__link**
  - Un **nav-bar__link** tiene una variante **nav-bar__link—active** para mostrar cuando la ruta actual sea igual al item del menú y marcarlo de alguna forma
  - Se utiliza la librería **react-router-dom** para hacer la navegación

src/components/nav_bar.jsx

```
import React from 'react'
import { NavLink } from 'react-router-dom'

export const NavBar = () => {
	const navLinkClass = ({ isActive }) => {
		const classes = ['nav-bar__link']
		if (isActive) classes.push('nav-bar__link--active')
		return classes.join(' ')
	}
	return (
		<nav className='nav-bar'>
			<NavLink
				className={navLinkClass}
				to='/'
			>
				Inicio
			</NavLink>
			<NavLink
				className={navLinkClass}
				to='/lightbulb'
			>
				Ampolleta
			</NavLink>
		</nav>
	)
}
export default NavBar
```

Proyecto de ejemplo

- Modificando el Layout para incorporar un ruteador

src/components/layout.jsx

Es necesario encerrar todo el layout en este Componente especial que es un Proveedor de rutas de **react-router-dom**

Y agregamos nuestro componente **<NavBar />** entre el título y la página

```
import React from 'react'
import { BrowserRouter } from 'react-router-dom'

import NavBar from './nav_bar'

const Layout = () => {
        return (
                <BrowserRouter>
                        <div className='layout'>
                                <h1 className='layout__title'>Interfaz energética</h1>
                                <NavBar />
                                <div className='layout__page'>
                                        {/* Add Page here */}
                                        El contenido de la página va aquí
                                </div>
                        </div>
                </BrowserRouter>
        )
}

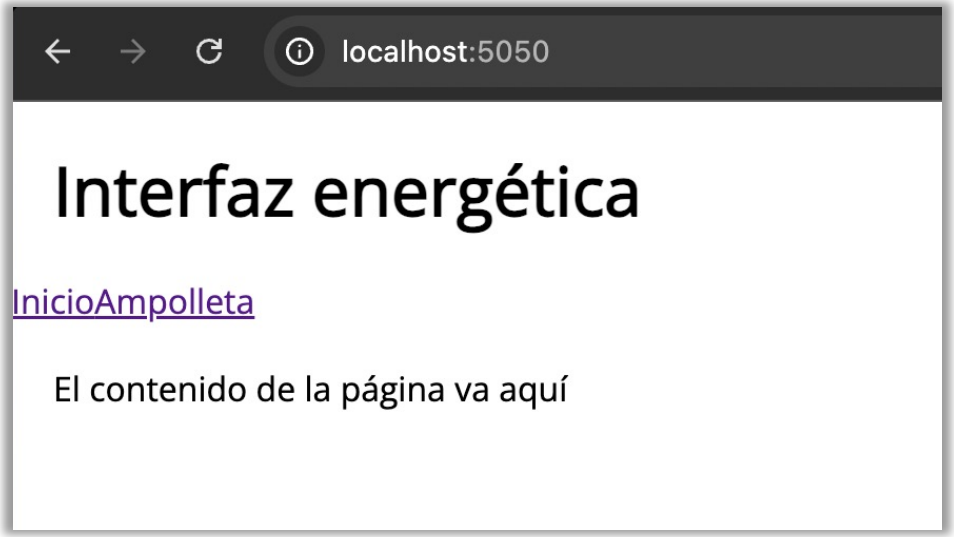
export default Layout
```

Más información sobre la librería para ruteador en https://reactrouter.com/en/main/start/tutorial

Proyecto de ejemplo

- Modificando el Layout para incorporar un ruteador

La página ahora se ve así, porque faltan estilos para el componente **<NavBar />**



Proyecto de ejemplo

- Componente NavBar (estilos)

Creamos estos archivos de estilos

src > stylesheets > nav-bar > index.scss

```scss
@import './nav-bar.scss';
@import './nav-bar__link.scss';
```

src > stylesheets > nav-bar > nav-bar.scss > ...

```scss
.nav-bar {
  display: flex;
  background-color: #ccc;
  padding-left: 20px;
}
```

src > stylesheets > nav-bar > nav-bar__link.scss > ...

```scss
.nav-bar__link {
  display: inline-block;
  margin-right: 15px;
  padding: 8px;
  color: rgb(4, 100, 253);

  &--active {
    background: #222;
    color: white;
  }
}
```

Y agregamos **nav-bar/index.scss** al archivo principal de estilos

src > stylesheets > index.scss > ...

```scss
@import url('https://fonts.googleapis.com/css2?family=Open+Sans&display=swap');

@import './layout/index.scss';
@import './nav-bar/index.scss';
```

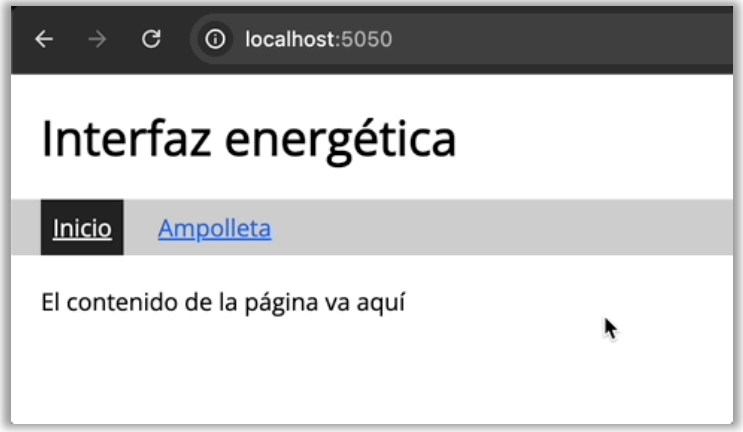
En este proyecto se utiliza SCSS que es un preprocesador de CSS que permite más facilidades para escribir estilos, como por ejemplo la anidación de estilos

Aprende más en https://sass-lang.com/

Proyecto de ejemplo

- Modificando el Layout para incorporar un ruteador

Si miramos el navegador, ahora nuestro proyecto se ve así



Los links son clickables y además modifican la url del navegador

Proyecto de ejemplo

- Modificando el Layout para incorporar un ruteador

Ahora vamos a mostrar un componente diferente dependiendo de que ruta esté mostrando el navegador, para eso agregamos otra modificación al componente **Layout**

Para eso usamos los componentes especiales **Routes** y **Route** de **react-router-dom**, los que nos permiten especificar que si la url (**path**) tiene determinado valor, entonces hay que mostrar el componente especificado en la propiedad **element**

Los componentes NavLink, Route y Routes necesariamente deben estar dentro de un BrowserRouter para funcionar

src/components/layout.jsx

```jsx
import React from 'react'
import { BrowserRouter, Routes, Route } from 'react-router-dom'

import HomePage from '../pages/home_page'
import LightbulbPage from '../pages/lightbulb_page'

import NavBar from '../components/nav_bar'

const Layout = () => {
        return (
                <BrowserRouter>
                        <div className='layout'>
                                <h1 className='layout__title'>Interfaz energética</h1>
                                <NavBar />
                                <div className='layout__page'>


                                        <Routes>
                                                <Route path='/' element={<HomePage />} />
                                                <Route path='/lightbulb' element={<LightbulbPage />} />
                                        </Routes>



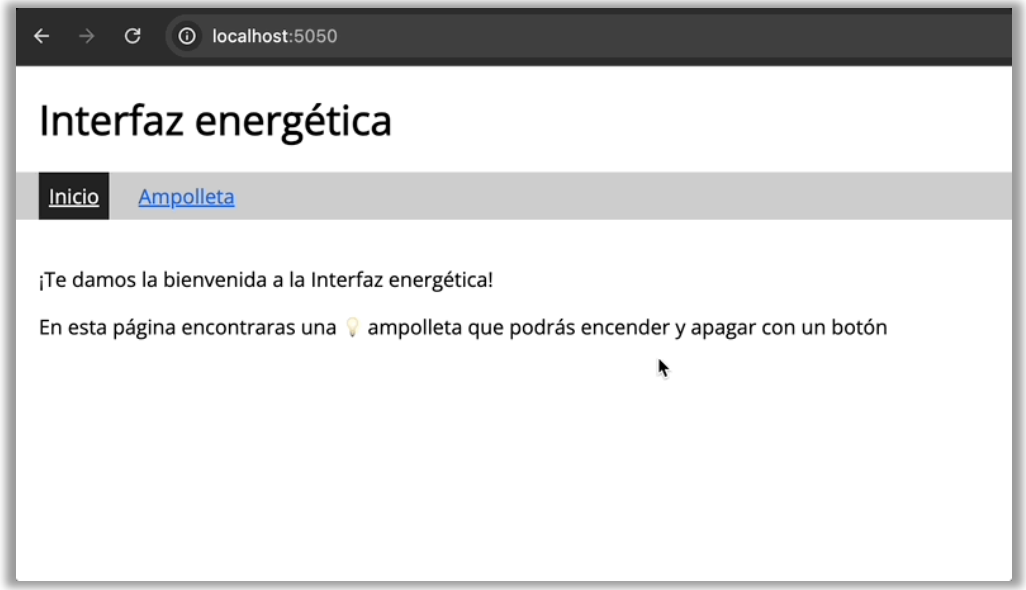
                                </div>
                        </div>
                </BrowserRouter>
        )
}

export default Layout
```

Proyecto de ejemplo

- Modificando el Layout para incorporar un ruteador

Si miramos el navegador, ahora nuestro proyecto se ve así



Proyecto de ejemplo

- Componente Página Ampolleta y usando un estado

src/pages/lightbulb_page.jsx

```jsx
import React, { useState } from 'react'

import LightBulbOff from '../assets/light-off.png'

export const LightbulbPage = () => {
	const [isOn, setIsOn] = useState(false);

	return (
		<div className='lightbulb-page'>
			<p>Presiona el botón para encender o apagar la ampolleta</p>
			<img src={LightBulbOff} height={130} alt='ampolleta' />
			<br />
			<br />

			<button onClick={() => setIsOn(!isOn)}>
				{isOn ? 'Apagar' : 'Encender'}
			</button>

		</div>
	)
}

export default LightbulbPage
```

useState es un hook de React que permite recordar el valor de una variable y cambiarlo durante la ejecución del código asegurando que la página se recargará automáticamente cuando su valor cambie

```
const [
    variable,
    setterValorVariable
] = useState(valorInicial);
```

Dentro de los corchetes van el nombre de la **variable** y el nombre de la función **setterValorVariable** que permite cambiar su valor

Dentro de **useState** va el **valorInicial** de la variable

https://es.react.dev/reference/react/hooks

Proyecto de ejemplo

- Componente Página Ampolleta y usando un estado

src/pages/lightbulb_page.jsx

```jsx
import React, { useState } from 'react'

import LightBulbOn from '../assets/light-on.png'
import LightBulbOff from '../assets/light-off.png'

export const LightbulbPage = () => {
	const [isOn, setIsOn] = useState(false);

	return (
		<div className='lightbulb-page'>
			<p>Presiona el botón para encender o apagar la ampolleta</p>
			<img src={isOn ? LightBulbOn : LightBulbOff} height={130}
			alt='ampolleta' />
			<br />
			<br />
			<button onClick={() => setIsOn(!isOn)}>
				{isOn ? 'Apagar' : 'Encender'}
			</button>
		</div>
	)
}

export default LightbulbPage
```



assets/light-off.png



assets/light-on.png

Proyecto de ejemplo

- Componente Página Ampolleta (estilos)

src > stylesheets > lightbulb-page > lightbulb-page.scss >

```scss
.lightbulb-page {
  button {
    width: 120px;
  }
}
```

src > stylesheets > lightbulb-page > index.scss

```scss
@import './lightbulb-page.scss';

```

src > stylesheets > index.scss > ...

```scss
@import url('https://fonts.googleapis.com/css2?family=Open+Sans&display=swap');

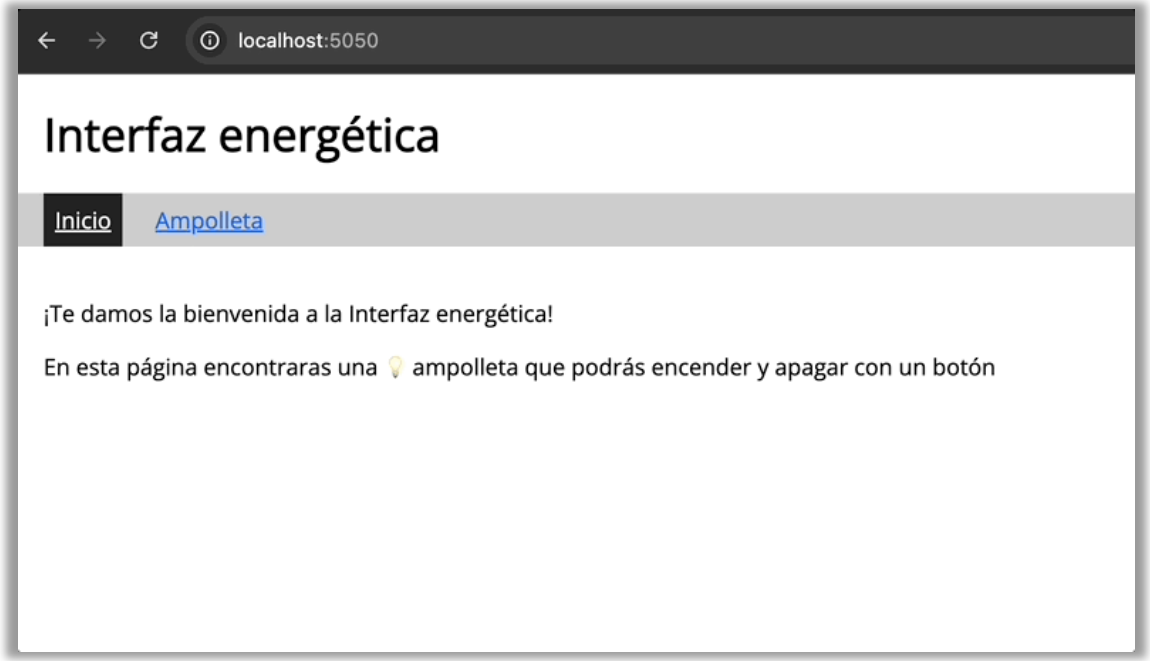
@import './layout/index.scss';
@import './nav-bar/index.scss';
@import './lightbulb-page/index.scss';

html, body {
  padding: 0;
  margin: 0;
}

* {
  font-family: 'Open Sans', sans-serif;
}
```

Proyecto de ejemplo

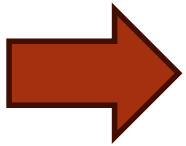
- Proyecto terminado en el navegador



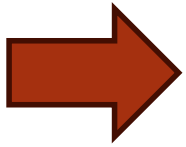
Proyecto de ejemplo

- Configuración de Webpack

La configuración de Webpack define cual es el archivo principal del proyecto



Webpack genera un archivo index.html con este título



webpack.config.js > ...

```
5   module.exports = (_, argv) => ({
6     entry: './src/index.js',
7     output: {
8       filename: '[name].[contenthash].js',
9       publicPath: '/'
10    },
11    plugins: [
12      new MiniCssExtractPlugin({
13        filename: '[name].[contenthash].css'
14      }),
15      new HtmlWebpackPlugin({
16        title: 'Interfaz energética'
17      })
18    ],
```

Proyecto de ejemplo

- Comandos del proyecto en **package.json**

```
{} package.json > ...
{
  "name": "frontend-sample-app",
  "version": "0.1.0",
  "description": "Basic Frontend Development Environment to start an app with Webpack",
  "main": "src/index.js",
  "repository": {
    "type": "git",
    "url": "https://github.com/alexarenasf/frontend-sample-app"
  },
  ▷ Debug
  "scripts": {
    "prod": "webpack --mode=production",
    "dev": "webpack serve --mode=development"
  },
```

Comando	Descripción
npm run dev	Inicia el modo desarrollo de Webpack
npm run prod	Webpack compila y transpila el proyecto con todos sus assets listo para ser desplegado en un servidor